Module 6 - Spring Data JPA with Spring Boot & Hibernate

1. Introduction to Spring Data JPA

Java introduced **JPA (Java Persistence API)**. JPA is a specification that defines a standard way to map Java objects to database tables. Instead of writing SQL for every operation, developers work with Java objects, while JPA handles most database interactions automatically.

Spring Data JPA builds on top of JPA and makes database access even easier. It provides ready-made repository interfaces, automatic query generation, pagination, sorting, and many other features. When used with **Spring Boot** and **Hibernate**, developers can build database-driven applications quickly with very little boilerplate code.

Instead of writing SQL like:

SELECT * FROM student;

developers simply work with Java objects.

Example:

Student student = repository.findById(1).get();

The SQL query is generated automatically by JPA.

What is Hibernate?

Hibernate is the **most popular implementation of JPA**.

Hibernate converts Java objects into database records and vice versa.

What is Spring Data JPA?

Spring Data JPA is a Spring module that simplifies working with JPA.

Instead of writing DAO classes and SQL queries manually, developers simply create a Repository interface.

Example:

public interface StudentRepository extends JpaRepository<Student, Integer> {

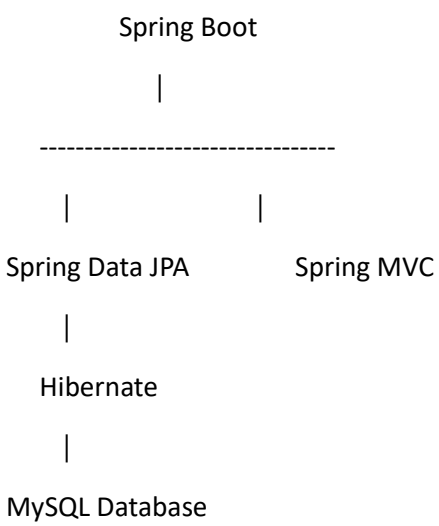
}

That's enough to get methods like

- save()
- findAll()
- findById()
- delete()
- count()

without writing any SQL.

Relationship between Spring Boot, Spring Data JPA and Hibernate



Why use Spring Data JPA?

Without Spring Data JPA, developers need to:

- Open database connections.
- Write SQL queries.
- Execute SQL.
- Convert ResultSet into Java objects.
- Close connections.

Spring Data JPA performs all these tasks automatically, allowing developers to focus on business logic.

JDBC vs JPA vs Spring Data JPA

Feature	**JDBC**	**JPA**	**Spring Data JPA**
SQL Writing	Manual	Partial	Mostly Automatic
Boilerplate Code	High	Medium	Very Low
ORM Support	No	Yes	Yes
CRUD Operations	Manual	Automatic	Automatic
Query Generation	Manual	Limited	Automatic
Learning Curve	Easy	Medium	Easy

How Spring Data JPA Works

User –> Controller –> Service -> Repository (JpaRepository) –> Hibernate -> MySQL Database

Example

Imagine a college management system.

Without Spring Data JPA, to save a student you would:

1. Open a database connection.
2. Write an INSERT SQL statement.
3. Execute the SQL.

4. Close the connection.

With Spring Data JPA, saving a student is simply:

studentRepository.save(student);

Everything else is handled automatically.

Spring Data JPA Example

---- Save one student into a MySQL database.

Student.java

```
package org.example.studentjpa.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "students")
public class Student {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Integer id;
   private String name;
   private int age;
   public Student() {
   }
   public Student(String name, int age) {
      this.name = name;
      this.age = age;
   }
   public Integer getId() {
      return id;
   }
   public void setId(Integer id) {
      this.id = id;
   }
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
```

```
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}
```

StudentRepository.java

```
package org.example.studentjpa.repository;
import org.example.studentjpa.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepository extends JpaRepository<Student, Integer> {
}
```

Runner.java

```
package org.example.studentjpa;
import org.example.studentjpa.entity.Student;
import org.example.studentjpa.repository.StudentRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Runner implements CommandLineRunner {
    private final StudentRepository repository;
    public Runner(StudentRepository repository) {
        this.repository = repository;
    }
    @Override
    public void run(String... args) {
        Student student = new Student("Yagna", 21);
        repository.save(student);
        System.out.println("Student Saved Successfully!");
    }
}
```

StudentJpaApplication.java

```
package org.example.studentjpa;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class StudentJpaApplication {

   public static void main(String[] args) {

      SpringApplication.run(StudentJpaApplication.class, args);

   }

}
```

Expected Output

Console:

Hibernate:

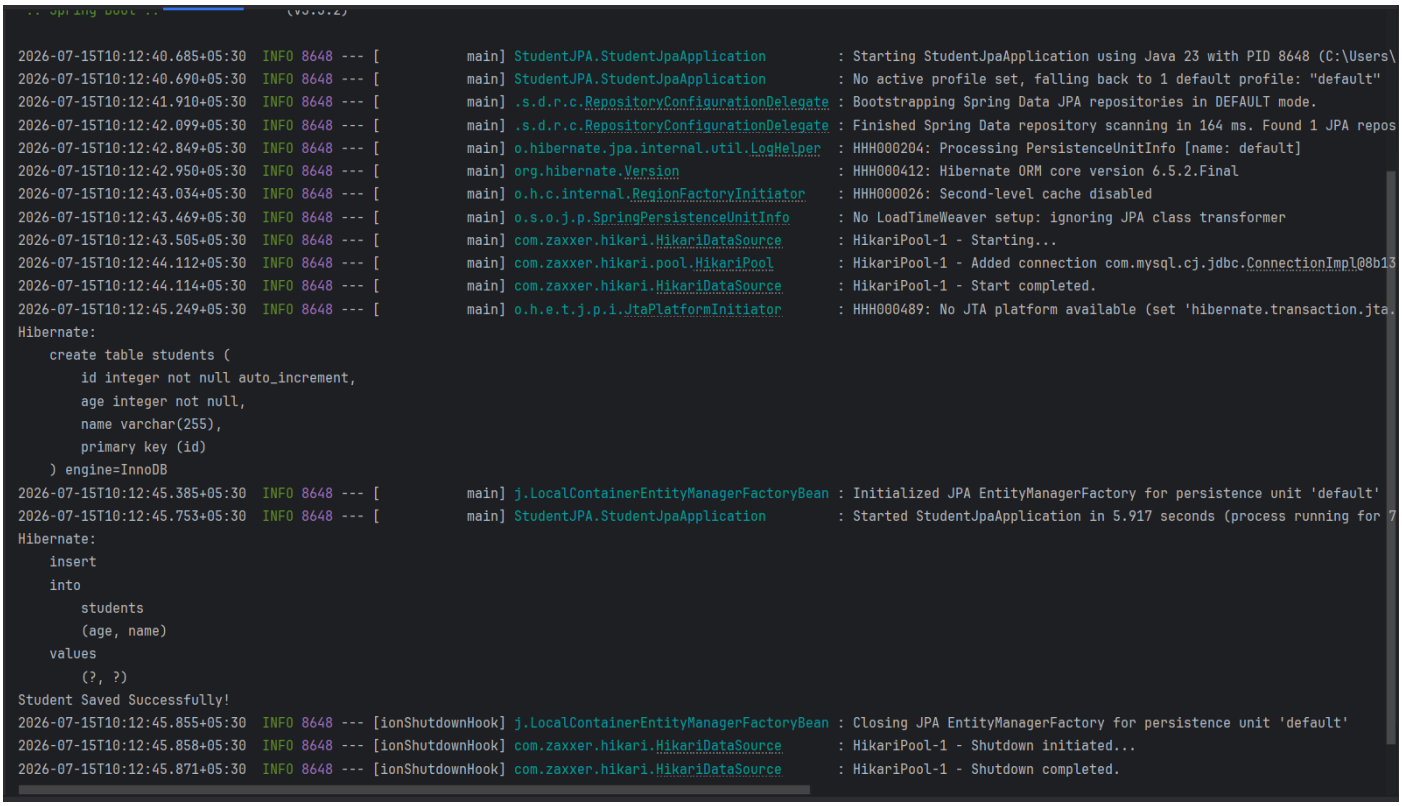
insert into students (age,name) values (?,?)

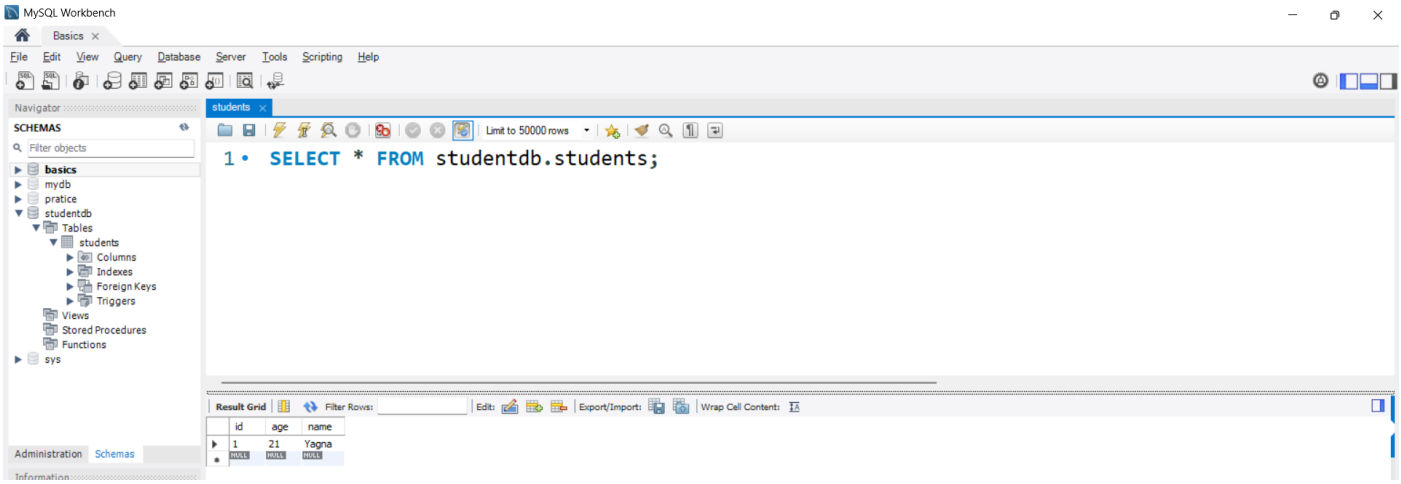
Student Saved Successfully!

MySQL Table (students):

id	name	age
1	Yagna	21

Output:





Setting Up a Spring Boot Project with Spring Data JPA

Before developing a Spring Data JPA application, it is necessary to configure the project correctly. Spring Boot simplifies this process by providing starter dependencies, automatic configuration, and embedded server support. Developers only need to add the required dependencies in the pom.xml file and configure the database connection in the application.properties file.

In a Spring Boot application, Maven manages all project dependencies. Instead of downloading multiple JAR files manually, Maven automatically retrieves the required libraries from the Maven Central Repository. Once the dependencies are added, Spring Boot automatically configures the application based on the available libraries.

To communicate with a database, Spring Boot requires database connection details such as the database URL, username, password, and Hibernate configuration. These properties are stored in the application.properties file.

Step 1: Create a Spring Boot Project

You can create a Spring Boot project in **IntelliJ IDEA** or by using **Spring Initializr**.

Using Spring Initializr

Select the following options:

OPTION	VALUE
PROJECT	Maven
LANGUAGE	Java
SPRING BOOT	Latest Stable Version
GROUP	org.example
ARTIFACT	StudentManagement
PACKAGING	Jar
JAVA VERSION	17

Required Dependencies

Add the following dependencies while creating the project:

- Spring Web
- Spring Data JPA
- MySQL Driver

These dependencies provide web development support, database operations, and MySQL connectivity.

Step 2: Project Structure

```
StudentManagement
|
├── pom.xml
|
└── src
    └── main
        ├── java
        |   └── org.example.studentmanagement
        |       ├── StudentManagementApplication.java
        |       ├── controller
        |       ├── service
        |       ├── repository
        |       ├── entity
        |       └── config
        |
        └── resources
            ├── application.properties
            ├── static
            └── templates
```

Step 3: pom.xml

The pom.xml file stores all project dependencies.

```
<dependencies>
    <!-- Spring Boot Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <!-- Spring Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
```

```
    <!-- MySQL -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
    </dependency>
    <!-- Testing -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Explanation of Dependencies

spring-boot-starter-web

Provides support for:

- Spring MVC
- REST APIs
- Embedded Tomcat

spring-boot-starter-data-jpa

Provides:

- JPA
- Hibernate
- Repository support
- Transaction management

mysql-connector-j

Allows the application to connect with a MySQL database.

spring-boot-starter-test

Provides:

- JUnit
- Mockito
- Spring Test Framework

Step 4: Create MySQL Database

Open MySQL Workbench and execute:

```
CREATE DATABASE studentdb;
```

Step 5: Configure application.properties

Create the file:

src/main/resources/application.properties

Add the following configuration:

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb

spring.datasource.username=root

spring.datasource.password=YOUR_PASSWORD

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

Replace

YOUR_PASSWORD

with your MySQL password.

Explanation

Database URL

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb

Connects the application to the MySQL database named studentdb.

Username

spring.datasource.username=root

Specifies the MySQL username.

Password

spring.datasource.password=password

Specifies the MySQL password.

Driver

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

Loads the MySQL JDBC driver.

ddl-auto

spring.jpa.hibernate.ddl-auto=update

Automatically creates or updates database tables based on entity classes.

Common values:

Value	Purpose
create	Creates new tables every time.
update	Updates existing tables.

validate	Checks table structure only.
none	No action.

Show SQL

spring.jpa.show-sql=true

Displays generated SQL queries in the console.

Format SQL

spring.jpa.properties.hibernate.format_sql=true

Formats SQL queries for better readability.

Step 6: Main Class

```
package org.exmple.studentmanagement;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class StudentManagementApplication {

   public static void main(String[] args) {

      SpringApplication.run(StudentManagementApplication.class, args);

   }

}
```

Running the Application

Run:

StudentManagementApplication.java

If everything is configured correctly, the console displays:

Tomcat started on port 8080

Started StudentManagementApplication

Hibernate also connects to MySQL and creates the required tables automatically.

Entity Mapping

In Spring Data JPA, an **Entity** is a Java class that represents a table in a relational database. Every object of the entity class corresponds to one row in the database table, while each field in the class corresponds to a column. Instead of manually writing SQL statements to create tables and insert data, developers simply define an entity class using JPA annotations, and Hibernate automatically generates the corresponding table.

Entity mapping is one of the most important features of JPA because it establishes the relationship between Java objects and database tables. The mapping is achieved using annotations such as @Entity, @Table, @Id, @GeneratedValue, and @Column. These annotations provide instructions to Hibernate about how the class should be stored in the database.

What is an Entity?

An **Entity** is a Plain Old Java Object (POJO) that is managed by JPA.

For example,

Student.java

students table

Each object represents one record.

Example:

id	name	age
1	Yagna	21
2	Rahul	20

becomes

Student s1 = new Student(1,"Yagna",21);

Student s2 = new Student(2,"Rahul",20);

Common JPA Annotations

Annotation	Purpose
@Entity	Marks the class as a database entity.
@Table	Specifies the database table name.
@Id	Defines the primary key.
@GeneratedValue	Generates primary key values automatically.
@Column	Maps a field to a table column.
@Transient	Ignores a field (not stored in the database).

Explanation of Annotations

@Entity

Marks the Java class as a database entity.

```
@Entity
public class Student{


}
```

Without this annotation, Hibernate ignores the class.

@Table

Specifies the database table name.

@Table(name="students")

Without it, Hibernate uses the class name (Student) as the table name by default.

@Id

Defines the primary key.

@Id

private Integer id;

Every entity **must have exactly one primary key**.

@GeneratedValue

Automatically generates primary key values.

@GeneratedValue(strategy =

GenerationType.IDENTITY)

Suppose you insert three students.

Hibernate automatically generates:

1

2

3

You don't have to assign IDs manually.

@Column

Maps a field to a database column.

Example

@Column(nullable = false)

private String name;

Now the database will not allow:

NULL

values for the name column.

Generated Database Table

Hibernate creates:

CREATE TABLE students(

id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(255) NOT NULL,

age INT NOT NULL

);

Notice:

You never wrote this SQL.

Hibernate generated it automatically.

Spring Data Repositories

A **Repository** in Spring Data JPA acts as the bridge between the application and the database. It is responsible for performing data access operations such as inserting, updating, deleting, and retrieving records. Instead of writing Data Access Object (DAO) classes manually, Spring Data JPA provides ready-made repository interfaces that automatically implement common database operations.

Spring Data JPA provides several repository interfaces, each offering different levels of functionality. The most commonly used interface is **JpaRepository**, which includes CRUD operations, pagination, sorting, and many advanced features.

Types of Repository Interfaces

Repository	Purpose
Repository	Marker interface (base interface).
CrudRepository	Provides CRUD operations.
PagingAndSortingRepository	Adds pagination and sorting.
JpaRepository	Provides CRUD, pagination, sorting, batch operations, and JPA-specific features.

Common JpaRepository Methods

Method	Purpose
save()	Insert or update a record
findAll()	Retrieve all records
findById()	Retrieve a record by ID
deleteById()	Delete a record by ID
delete()	Delete an object
existsById()	Check whether a record exists
count()	Count total records

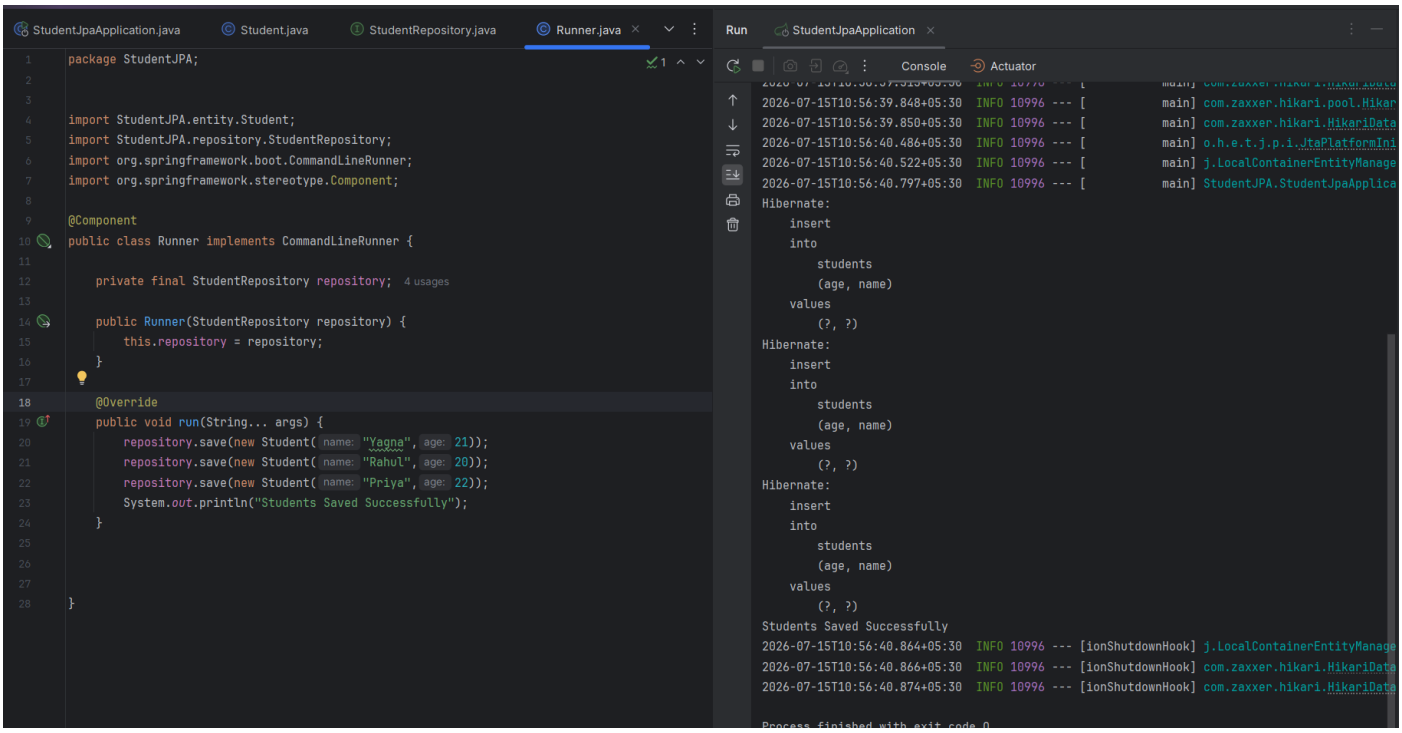
Example 1: Save Student

Runner.java

```
package org.example.studentmanagement;

import org.example.studentmanagement.entity.Student;

import org.example.studentmanagement.repository.StudentRepository;

import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype.Component;

@Component

public class Runner implements CommandLineRunner {
```

```
    private final StudentRepository repository;
    public Runner(StudentRepository repository) {
        this.repository = repository;
    }
    @Override
    public void run(String... args) {
        repository.save(new Student("Yagna",21));
        repository.save(new Student("Rahul",20));
        repository.save(new Student("Priya",22));
        System.out.println("Students Saved Successfully");
    }
}
```

Output



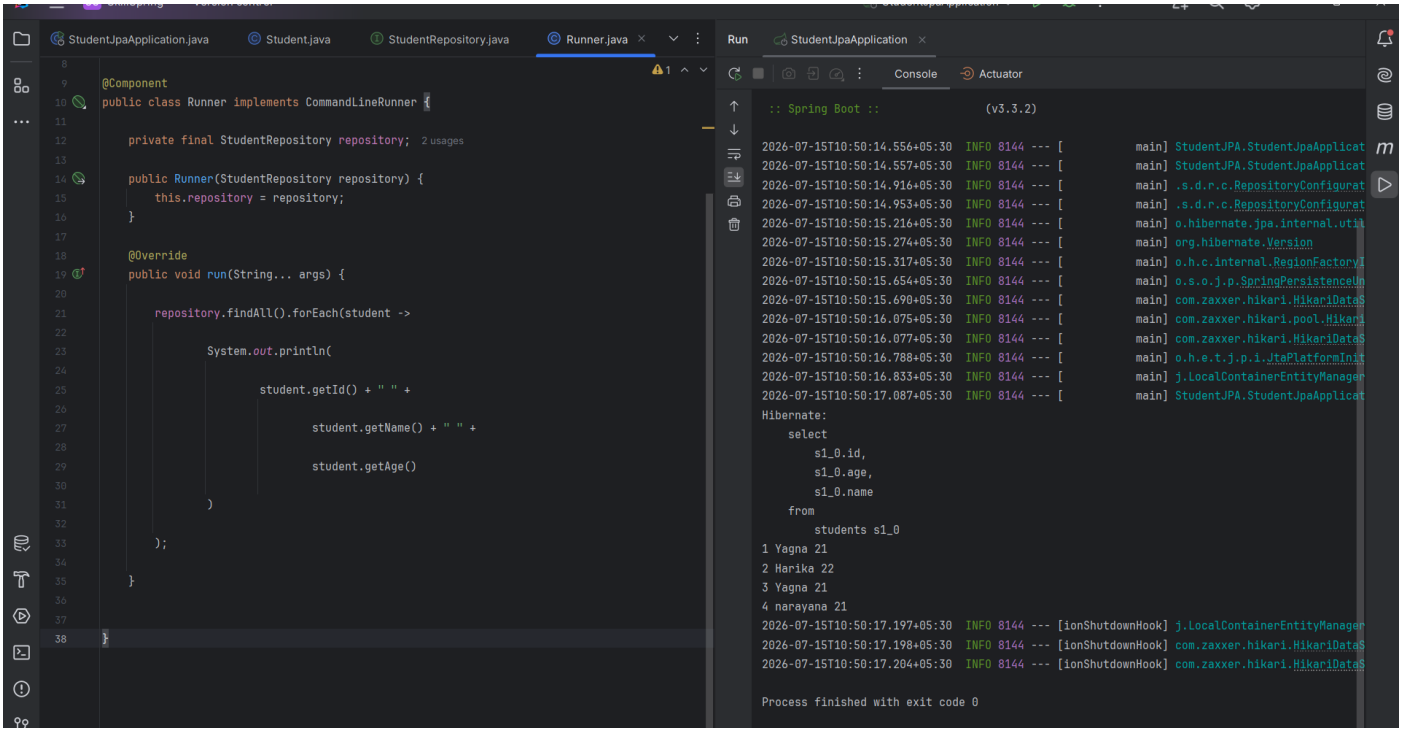
Example 2: Find All Students

Replace the run() method with:

```
@Override
public void run(String... args) {
    repository.findAll().forEach(student ->
            System.out.println(
                    student.getId() + " " +
                    student.getName() + " " +
                    student.getAge()
```

```
        )
    );
}
```

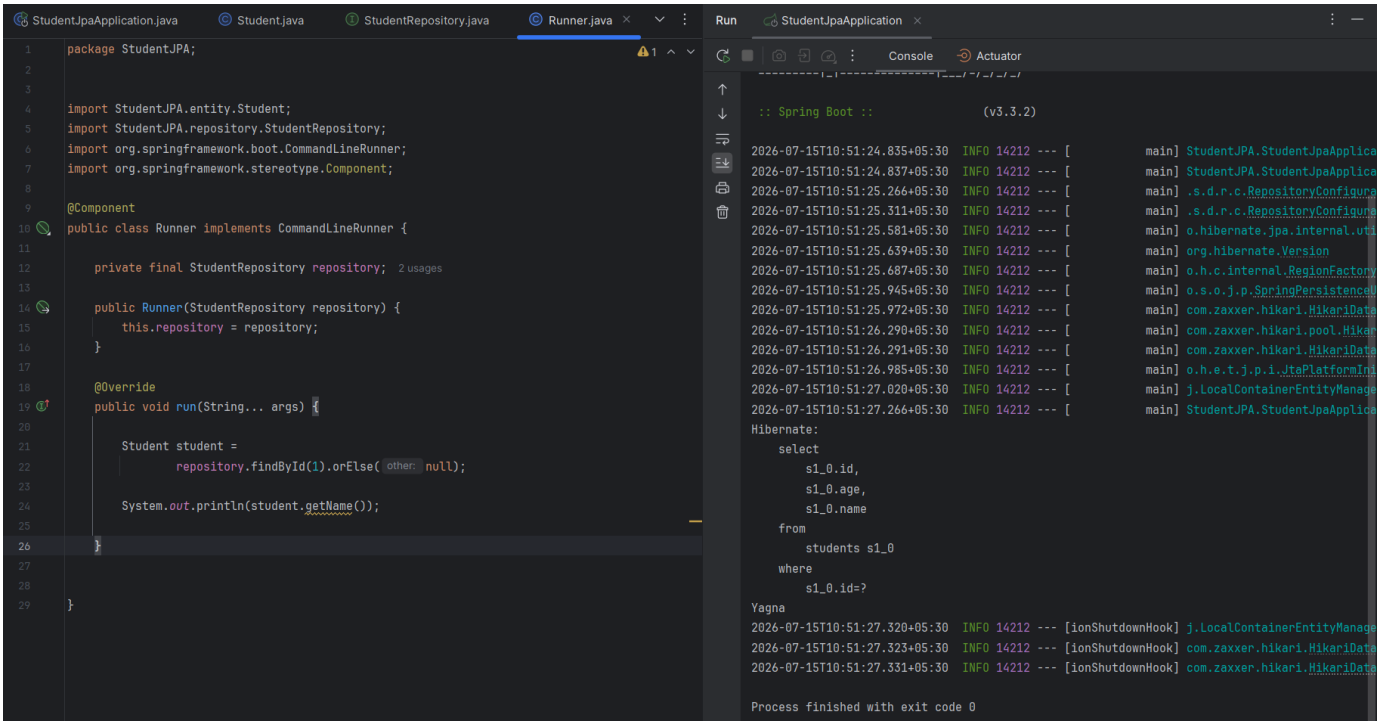
Output



Example 3: Find Student by ID

```
@Override
public void run(String... args) {
    Student student =
            repository.findById(1).orElse(null);
    System.out.println(student.getName());
}
```

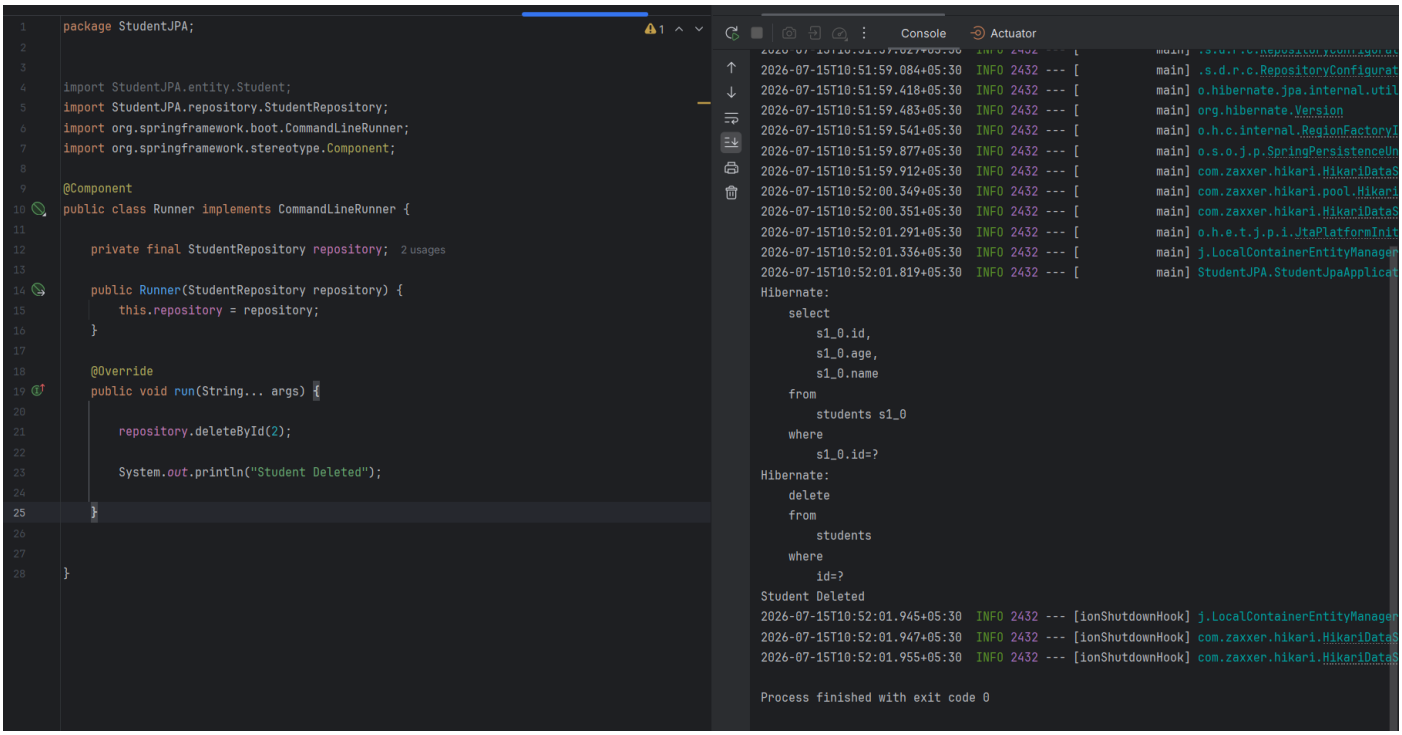
Output



Example 4: Delete Student

```
@Override
public void run(String... args) {
   repository.deleteById(2);
   System.out.println("Student Deleted");
}
```

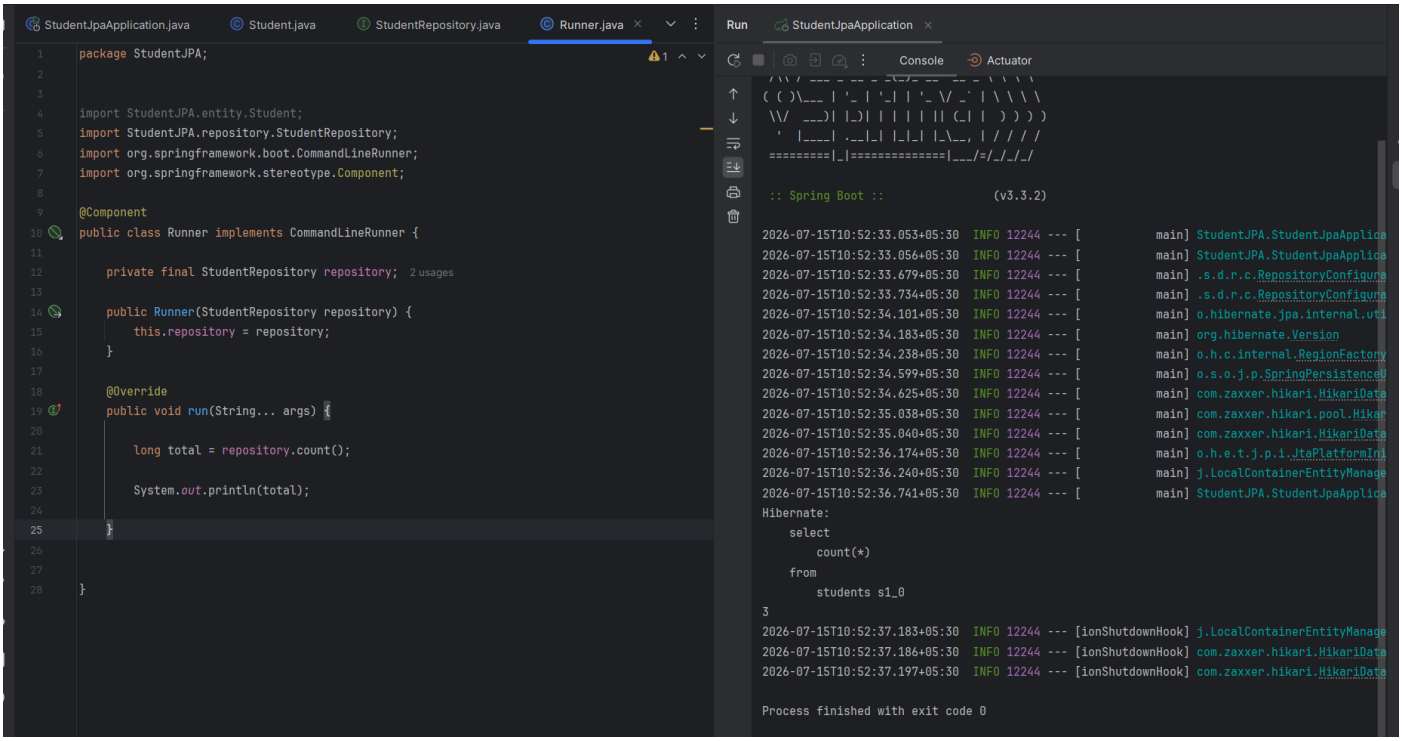
Output



Example 5: Count Students

```
@Override
public void run(String... args) {
    long total = repository.count();
    System.out.println(total);
}
```

Output



Custom Query Methods

Spring Data JPA can generate SQL automatically based on method names.

Example:

```
public interface StudentRepository
      extends JpaRepository<Student,Integer>{
   Student findByName(String name);
}
```

No SQL required.

Spring automatically generates:

SELECT * FROM students

WHERE name=?

Using @Query Annotation

Sometimes automatic query generation is not enough.

Spring provides the @Query annotation.

```
package org.example.studentmanagement.repository;
```

```
import org.example.studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
public interface StudentRepository
      extends JpaRepository<Student,Integer>{
   @Query("SELECT s FROM Student s WHERE s.age > :age")
   java.util.List<Student> findStudents(@Param("age") int age);
}
```

Calling the Query

```
@Override
public void run(String... args) {
   repository.findStudents(20)
         .forEach(s ->
               System.out.println(s.getName())
         );
}
```

Output

Yagna

Priya

Derived Query Methods

Spring Data JPA understands many keywords.

Method	SQL Generated
findByName()	WHERE name=?
findByAge()	WHERE age=?
findByNameContaining()	LIKE '%text%'
findByAgeGreaterThan()	age > ?
findByAgeLessThan()	age < ?
findByNameStartingWith()	LIKE 'A%'
findByNameEndingWith()	LIKE '%a'
findByAgeBetween()	BETWEEN

5. CRUD Operations with Spring Data JPA

CRUD stands for **Create, Read, Update, and Delete**, which are the four basic operations performed on database records. In Spring Data JPA, these operations are simplified through the JpaRepository interface, which provides built-in methods for managing entities.

In a typical Spring Boot application, CRUD operations follow a layered architecture. The **Controller** receives HTTP requests from the client, the **Service** contains business logic, the **Repository** communicates with the database, and **Hibernate** translates Java objects into SQL queries.

CRUD

HTTP Method	URL	Operation
POST	/students	Create Student
GET	/students	Get All Students
GET	/students/{id}	Get Student by ID
PUT	/students/{id}	Update Student
DELETE	/students/{id}	Delete Student

Student.java

```
package org.example.studentmanagement.entity;

import jakarta.persistence.*;

@Entity

@Table(name="students")

public class Student {

   @Id

   @GeneratedValue(strategy = GenerationType.IDENTITY)

   private Integer id;

   private String name;

   private int age;

   public Student() {

   }

   public Student(String name, int age) {

      this.name = name;

      this.age = age;

   }

   public Integer getId() {

      return id;

   }
```

```
    public void setId(Integer id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
}
```

StudentRepository.java

```
package org.example.studentmanagement.repository;
import org.example.studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepository
        extends JpaRepository<Student,Integer> {
}
```

StudentService.java

```
package org.example.studentmanagement.service;
import org.example.studentmanagement.entity.Student;
import org.example.studentmanagement.repository.StudentRepository;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class StudentService {
    private final StudentRepository repository;
    public StudentService(StudentRepository repository) {
        this.repository = repository;
```

```
    }
    // Create
    public Student saveStudent(Student student) {
        return repository.save(student);
    }
    // Read All
    public List<Student> getStudents() {
        return repository.findAll();
    }
    // Read By Id
    public Student getStudent(Integer id) {
        return repository.findById(id).orElse(null);
    }
    // Update
    public Student updateStudent(Integer id, Student newStudent) {
        Student student = repository.findById(id).orElse(null);
        if(student != null){
            student.setName(newStudent.getName());
            student.setAge(newStudent.getAge());
            return repository.save(student);
        }
        return null;
    }
    // Delete
    public void deleteStudent(Integer id){
        repository.deleteById(id);
    }
}
```

StudentController.java

```
package org.example.studentmanagement.controller;
import org.example.studentmanagement.entity.Student;
import org.example.studentmanagement.service.StudentService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

```
@RestController
@RequestMapping("/students")
public class StudentController {
   private final StudentService service;
   public StudentController(StudentService service) {
      this.service = service;
   }
   // CREATE
   @PostMapping
   public Student addStudent(@RequestBody Student student){
      return service.saveStudent(student);
   }
   // READALL
   @GetMapping
   public List<Student> getStudents(){
      return service.getStudents();
   }
   // READ BY ID
   @GetMapping("/{id}")
   public Student getStudent(@PathVariable Integer id){
      return service.getStudent(id);
   }
   // UPDATE
   @PutMapping("/{id}")
   public Student updateStudent(
         @PathVariable Integer id,
         @RequestBody Student student){
      return service.updateStudent(id,student);
   }
   // DELETE
   @DeleteMapping("/{id}")
   public String deleteStudent(
         @PathVariable Integer id){
      service.deleteStudent(id);
```

```
        return "Student Deleted Successfully";
    }
}
```

StudentManagementApplication.java

```
package org.example.studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;


@SpringBootApplication
public class StudentManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(
                StudentManagementApplication.class,
                args
        );
    }
}
```

Running the Project

Run:

StudentManagementApplication.java

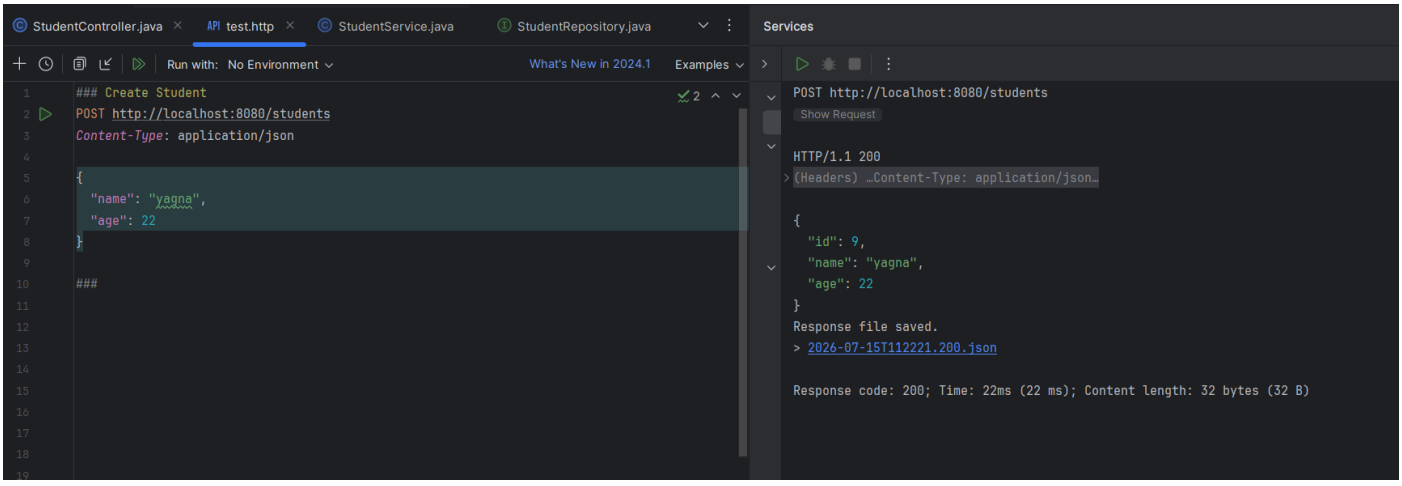
Tomcat starts on:

http://localhost:8080/students

Testing in ide by creating test.http

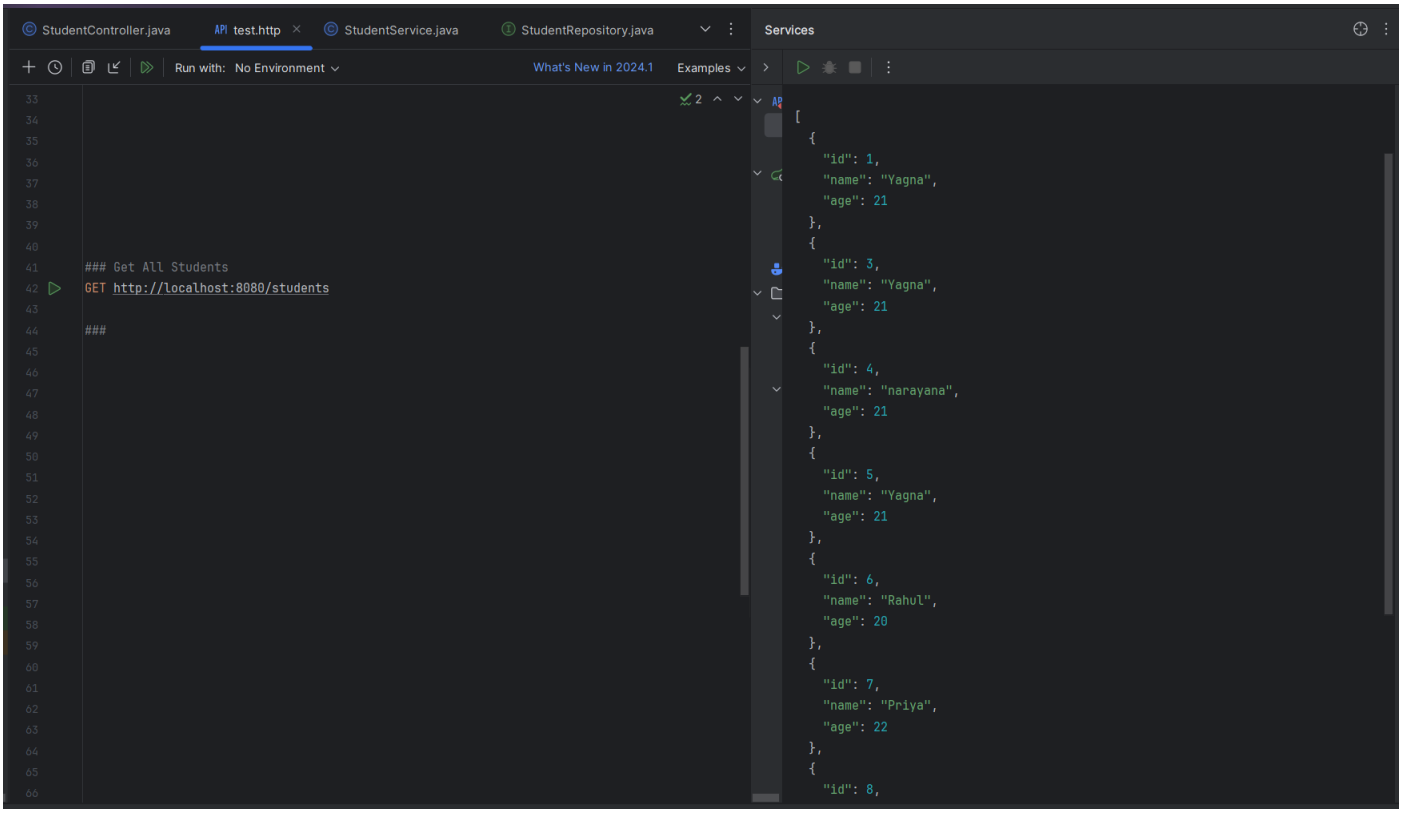
1 Create Student

POST



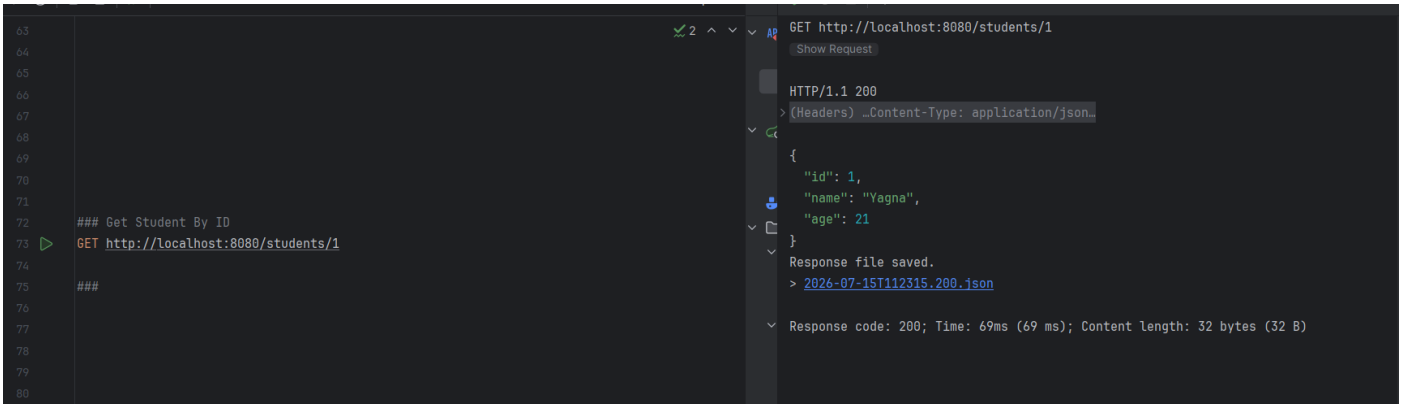
2 Get All Students

GET



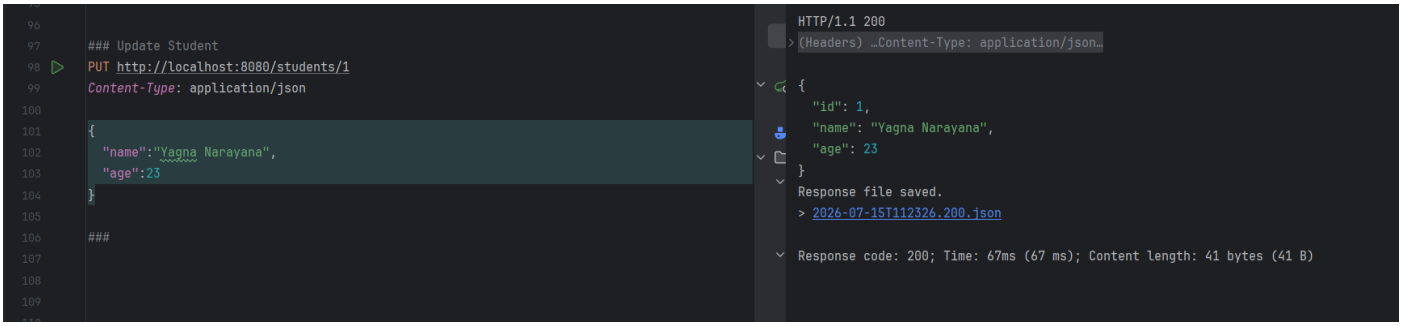
3 Get Student By ID

GET



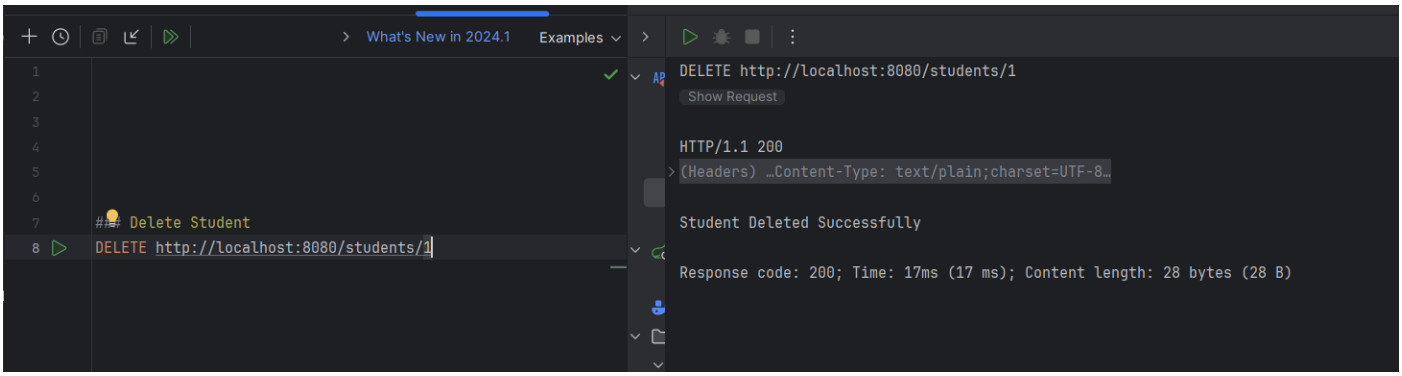
4 Update Student

PUT



5 Delete Student

DELETE



Query Methods and Named Queries

ISpring Data JPA provides powerful query capabilities without requiring developers to write SQL statements manually. One of its most useful features is **Query Methods**, where SQL queries are automatically generated based on the method names defined in the repository interface. This approach reduces development time and makes the code more readable.

For more complex database operations that cannot be expressed using method names alone, Spring Data JPA provides the **@Query annotation**, allowing developers to write custom JPQL or SQL queries. It also supports **Named Queries**, which are predefined queries associated with an entity class. These features make data retrieval flexible and efficient while maintaining clean and maintainable code.

Example:

StudentmanagementApplication.java

```
package Studentmanagement;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentManagementApplication {
   public static void main(String[] args) {
      SpringApplication.run(
            StudentManagementApplication.class,
            args
      );

   }
}
```

StudentController.java

```
package Studentmanagement.controller;
import Studentmanagement.entity.Student;
import Studentmanagement.service.StudentService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/students")
public class StudentController {
```

```
    private final StudentService service;
    public StudentController(StudentService service) {
        this.service = service;
    }
    // CREATE
    @PostMapping
    public Student addStudent(@RequestBody Student student){
        return service.saveStudent(student);

    }
    // READ ALL
    @GetMapping
    public List<Student> getStudents(){
        return service.getStudents();
    }
    // READ BY ID
    @GetMapping("/{id}")
    public Student getStudent(@PathVariable Integer id){
        return service.getStudent(id);
    }
    // UPDATE
    @PutMapping("/{id}")
    public Student updateStudent(
            @PathVariable Integer id,
            @RequestBody Student student){
        return service.updateStudent(id,student);
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String deleteStudent(
            @PathVariable Integer id){
        service.deleteStudent(id);
        return "Student Deleted Successfully";
    }
    @GetMapping("/name/{name}")
    public Student getStudentByName(@PathVariable String name){
        return service.getStudentByName(name);
    }
    @GetMapping("/hello")
    public String hello() {
        return "Hello Controller";
    }
}
```

```
package Studentmanagement.service;
import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class StudentService {
    private final StudentRepository repository;
```

```
    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }
    // Create
    public Student saveStudent(Student student) {
        return repository.save(student);
    }
    // Read All
    public List<Student> getStudents() {
        return repository.findAll();
    }

    // Read By Id
    public Student getStudent(Integer id) {
        return repository.findById(id).orElse(null);
    }
    // Update
    public Student updateStudent(Integer id, Student newStudent) {
        Student student = repository.findById(id).orElse(null);
        if(student != null){
            student.setName(newStudent.getName());
            student.setAge(newStudent.getAge());
            return repository.save(student);
        }
        return null;

    }
    // Delete
    public void deleteStudent(Integer id){
        repository.deleteById(id);

    }
    public Student getStudentByName(String name){
        return repository.findByName(name);
    }
}
```

StudentService.java

```
package Studentmanagement.service;
import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class StudentService {
    private final StudentRepository repository;
    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }
    // Create
    public Student saveStudent(Student student) {
        return repository.save(student);
    }
```

```
    // Read All
    public List<Student> getStudents() {
        return repository.findAll();
    }
    // Read By Id
    public Student getStudent(Integer id) {
        return repository.findById(id).orElse(null);
    }
    // Update
    public Student updateStudent(Integer id, Student newStudent) {
        Student student = repository.findById(id).orElse(null);
        if(student != null){
            student.setName(newStudent.getName());
            student.setAge(newStudent.getAge());
            return repository.save(student);
        }
        return null;
    }
    // Delete
    public void deleteStudent(Integer id){
        repository.deleteById(id);
    }
    public Student getStudentByName(String name){
        return repository.findByName(name);
    }
}
```

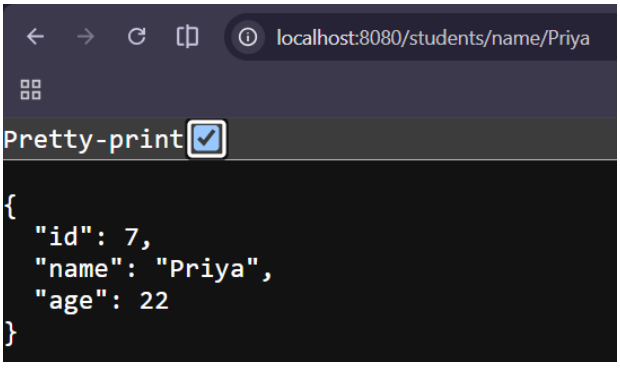
StudentRepository.java

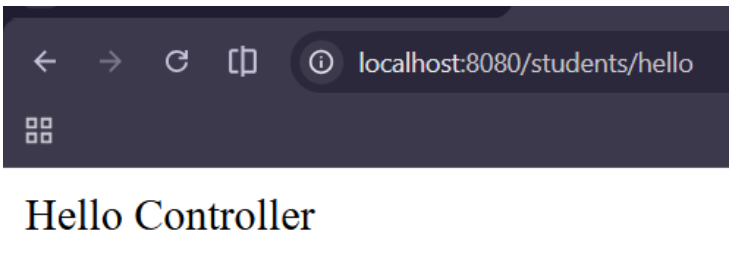
```
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student,Integer> {
    Student findByName(String name);
}
```

Student.java

```
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity
@Table(name="students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
```

```
    private int age;

    public Student() {
    }

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

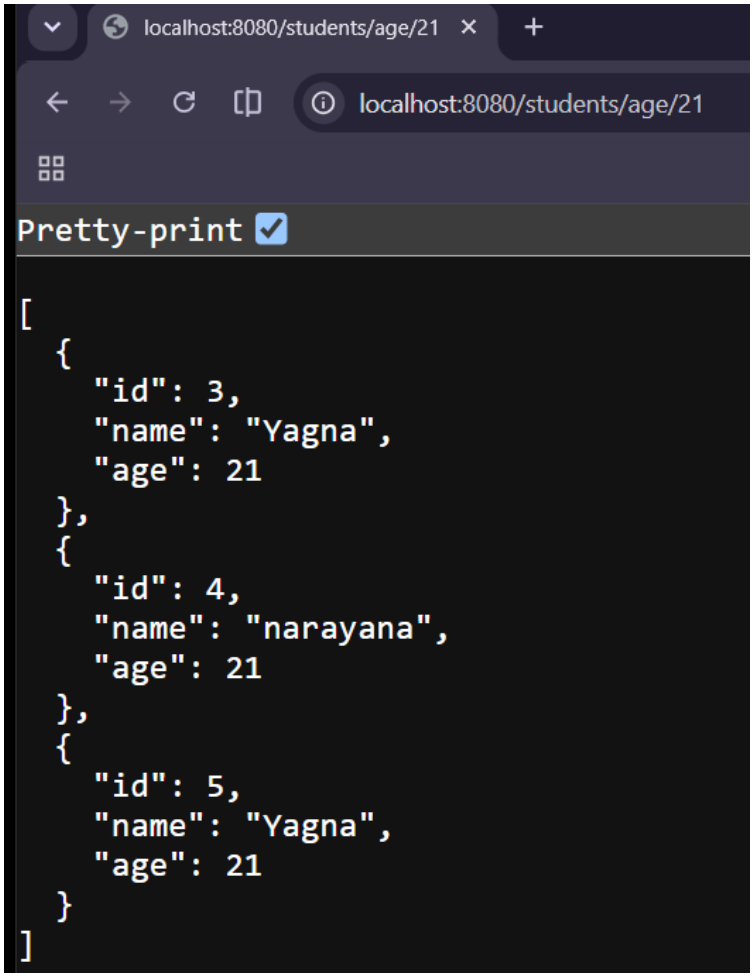




Multiple Query Methods

```
@GetMapping("/age/{age}")
public List<Student> getStudentsByAge(
        @PathVariable int age){
```

```
    return service.getStudentsByAge(age);
}
```

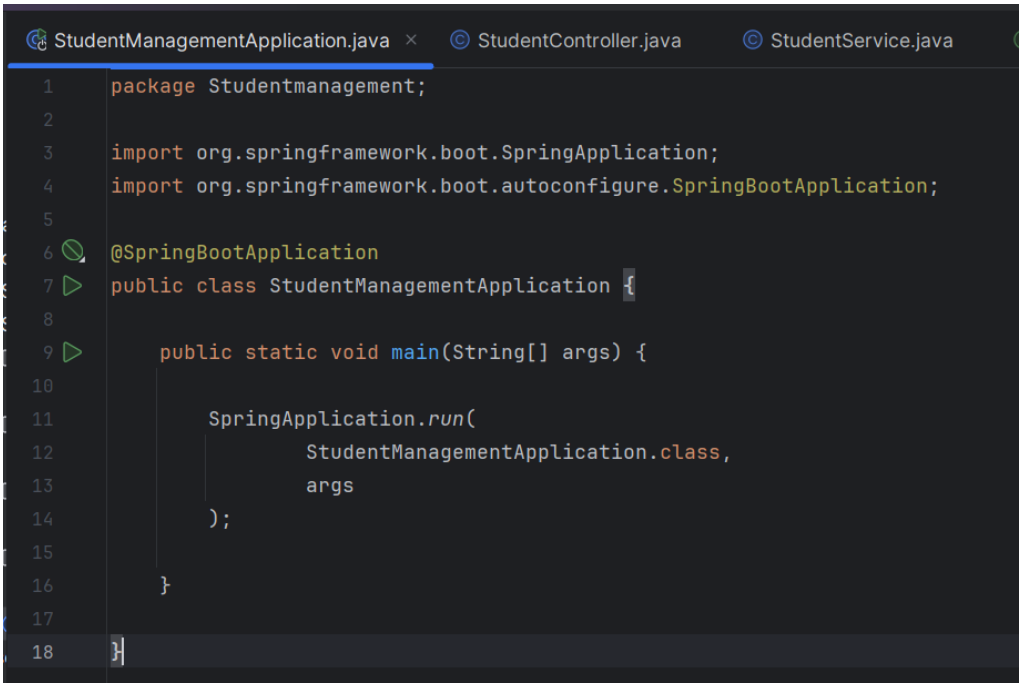


Common Query Keywords

Method	Description
findByName()	Find by name
findByAge()	Find by age
findByAgeGreaterThan()	Age greater than
findByAgeLessThan()	Age less than
findByAgeBetween()	Between two ages
findByNameContaining()	Name contains text
findByNameStartingWith()	Name starts with
findByNameEndingWith()	Name ends with
findByNameIgnoreCase()	Ignore uppercase/lowercase
findByAgeOrderByNameAsc()	Sort by name

Pagination and Sorting

Pagination divides the records into smaller pages, while **Sorting** arranges the records in ascending or descending order based on a specific field. These features improve performance and provide a better user experience.



StudentManagementApplication.java × | StudentController.java × | StudentService.java | Stud

```java
package Studentmanagement.controller;

import Studentmanagement.entity.Student;
import Studentmanagement.service.StudentService;
import org.springframework.web.bind.annotation.*;
import org.springframework.data.domain.Page;


@RestController
@RequestMapping("/students")

public class StudentController {

    private final StudentService service;  2 usages

    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping("/page")

    public Page<Student> getStudentsByPage(

            @RequestParam int page,
            @RequestParam int size){

        return service.getStudentsByPage(page,size);

    }
}
```

StudentManagementApplication.java | StudentController.java | StudentService.java ×

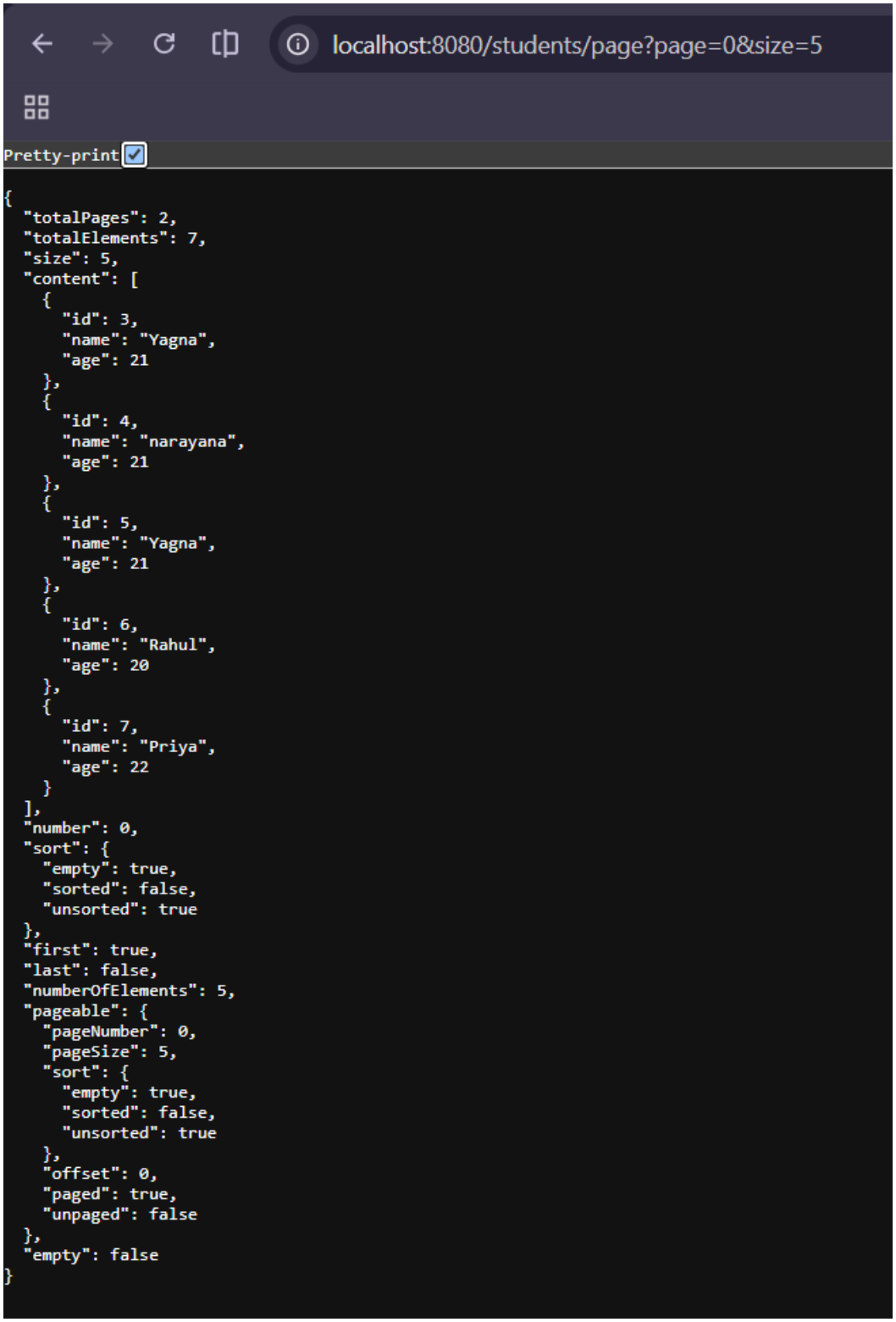
```java
package Studentmanagement.service;

import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.stereotype.Service;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;

@Service  3 usages
public class StudentService {

    private final StudentRepository repository;  2 usages

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public Page<Student> getStudentsByPage(int page,int size){  no usages

        return repository.findAll(PageRequest.of(page,size));

    }
}
```

oller.java | StudentService.java | StudentRepository.java × | Student.java

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;


public interface StudentRepository  3 usages
        extends JpaRepository<Student,Integer> {

}
```

roller.java | StudentService.java | StudentRepository.java | Student.java

```java
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity  6 usages
@Table(name="students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;  1 usage

    private int age;  1 usage

    public Student() {
    }

    public Student(String name, int age) {  4 usages
        this.name = name;
        this.age = age;
    }
}
```



Sorting:

Service

Ascending order

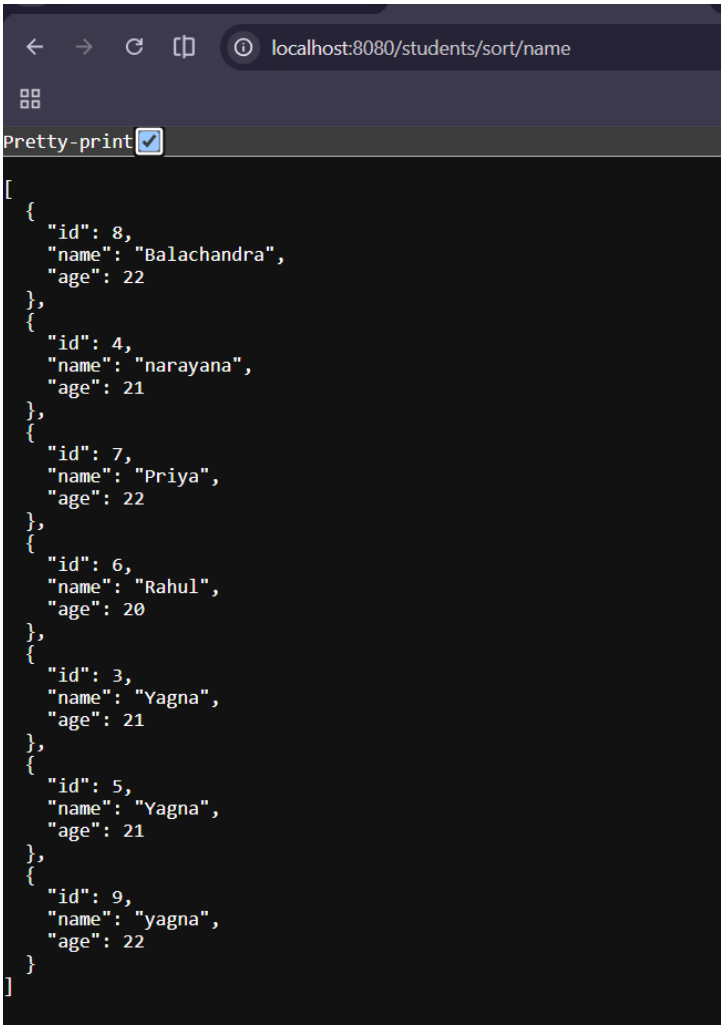
```
import org.springframework.data.domain.Sort;

public List<Student> sortByName(){
   return repository.findAll(
         Sort.by("name")
   );
```

}

Controller

```
@GetMapping("/sort/name")
public List<Student> sortByName(){
    return service.sortByName();
}
```

Output:



Descending Order

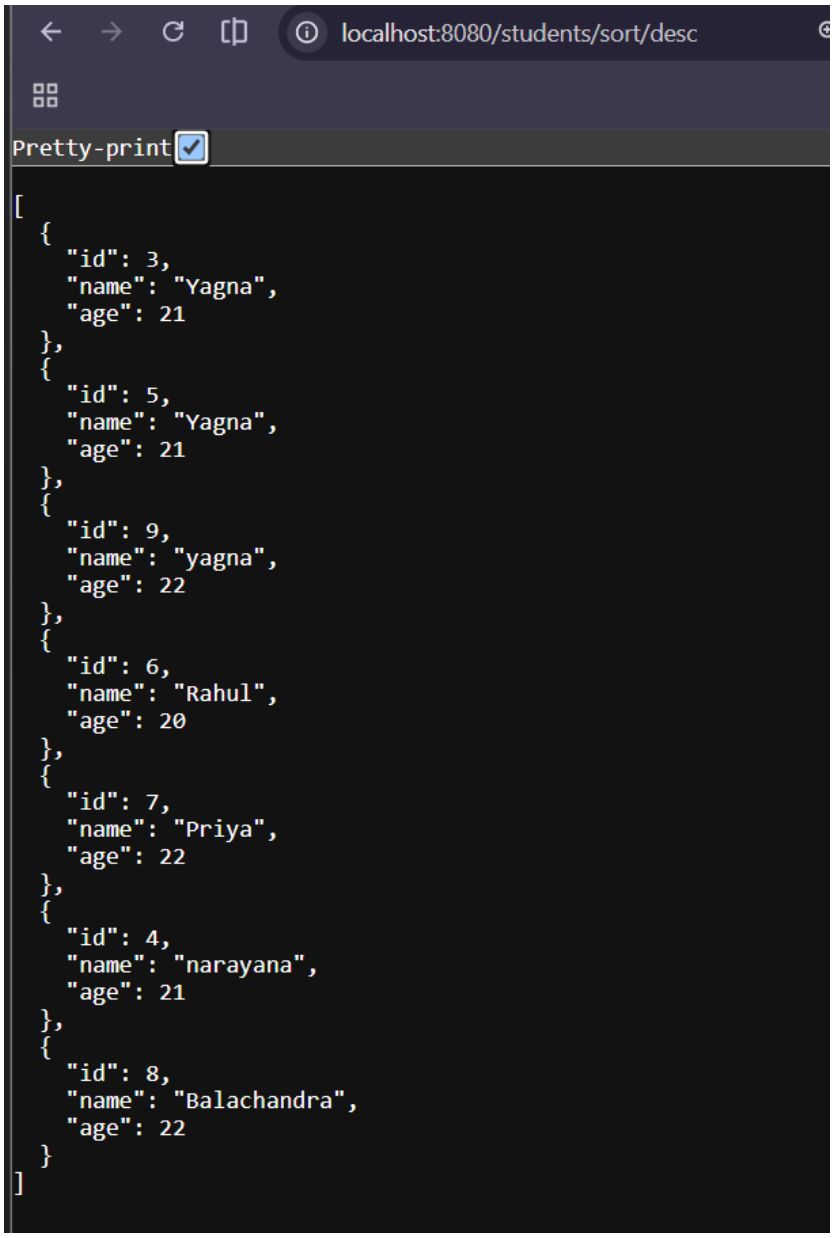
Service

```
public List<Student> sortDescending(){
    return repository.findAll(
            Sort.by(
                Sort.Direction.DESC,
                "name"
            )
    );
}
```

Controller

```
@GetMapping("/sort/desc")
public List<Student> sortDescending(){
   return service.sortDescending();
}
```

Output:



Auditing with Spring Data JPA

In real-world applications such as banking systems, hospital management systems, e-commerce platforms, and employee management systems, it is important to know **when a record was created**, **when it was last modified**, and sometimes **who created or modified it**. Instead of manually updating these fields every time a record changes, Spring Data JPA provides an **Auditing** feature.

Auditing automatically maintains these details using annotations such as @CreatedDate, @LastModifiedDate, @CreatedBy, and @LastModifiedBy. This reduces coding effort, improves accuracy, and helps maintain a complete history of database records.

Example:

StudentmanagementApplication.java

```
package Studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@SpringBootApplication
@EnableJpaAuditing
public class StudentManagementApplication {

   public static void main(String[] args) {

      SpringApplication.run(StudentManagementApplication.class,args);

   }

}
```

StudentController.java

```
package Studentmanagement.controller;

import Studentmanagement.entity.Student;
import Studentmanagement.service.StudentService;

import org.springframework.data.domain.Page;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")

public class StudentController {

   private final StudentService service;

   public StudentController(StudentService service){

      this.service=service;

   }

   // CREATE

   @PostMapping

   public Student addStudent(@RequestBody Student student){

      return service.saveStudent(student);

   }
```

```
// READ ALL

@GetMapping

public List<Student> getStudents(){

   return service.getStudents();

}

// READ BY ID

@GetMapping("/{id}")

public Student getStudent(@PathVariable Integer id){

   return service.getStudent(id);

}

// UPDATE

@PutMapping("/{id}")

public Student updateStudent(
      @PathVariable Integer id,
      @RequestBody Student student){

   return service.updateStudent(id,student);

}

// DELETE

@DeleteMapping("/{id}")

public String deleteStudent(@PathVariable Integer id){

   service.deleteStudent(id);

   return "Student Deleted Successfully";

}

// FIND BY NAME

@GetMapping("/name/{name}")

public List<Student> getStudentByName(@PathVariable String name){

   return service.getStudentByName(name);
```

```
    }

    // PAGINATION

    @GetMapping("/page")

    public Page<Student> getStudentsByPage(
            @RequestParam int page,
            @RequestParam int size){

        return service.getStudentsByPage(page,size);

    }

    // SORT ASCENDING

    @GetMapping("/sort/name")

    public List<Student> sortByName(){

        return service.sortByName();

    }

    // SORT DESCENDING

    @GetMapping("/sort/desc")

    public List<Student> sortDescending(){

        return service.sortDescending();

    }

}
```

StudentService.java

```
package Studentmanagement.service;

import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;

import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class StudentService {
```

```
private final StudentRepository repository;

public StudentService(StudentRepository repository){
   this.repository=repository;
}

// CREATE

public Student saveStudent(Student student){

   return repository.save(student);

}

// READ ALL

public List<Student> getStudents(){

   return repository.findAll();

}

// READ BY ID

public Student getStudent(Integer id){

   return repository.findById(id).orElse(null);

}

// UPDATE

public Student updateStudent(Integer id,Student newStudent){

   Student student=repository.findById(id).orElse(null);

   if(student!=null){

      student.setName(newStudent.getName());
      student.setAge(newStudent.getAge());

      return repository.save(student);

   }

   return null;

}

// DELETE

public void deleteStudent(Integer id){
```

```
        repository.deleteById(id);
    }
    // FIND BY NAME
    public List<Student> getStudentByName(String name){
        return repository.findByNameIgnoreCase(name);
    }
    // PAGINATION
    public Page<Student> getStudentsByPage(int page,int size){
        return repository.findAll(PageRequest.of(page,size));
    }
    // SORT ASCENDING
    public List<Student> sortByName(){
        return repository.findAll(Sort.by("name"));
    }
    // SORT DESCENDING
    public List<Student> sortDescending(){
        return repository.findAll(
                Sort.by(
                        Sort.Direction.DESC,
                        "name"
                )
        );
    }
}
```

StudentRepository.java

```
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
```

```
public interface StudentRepository extends JpaRepository<Student,Integer>{

   List<Student> findByNameIgnoreCase(String name);

}
```

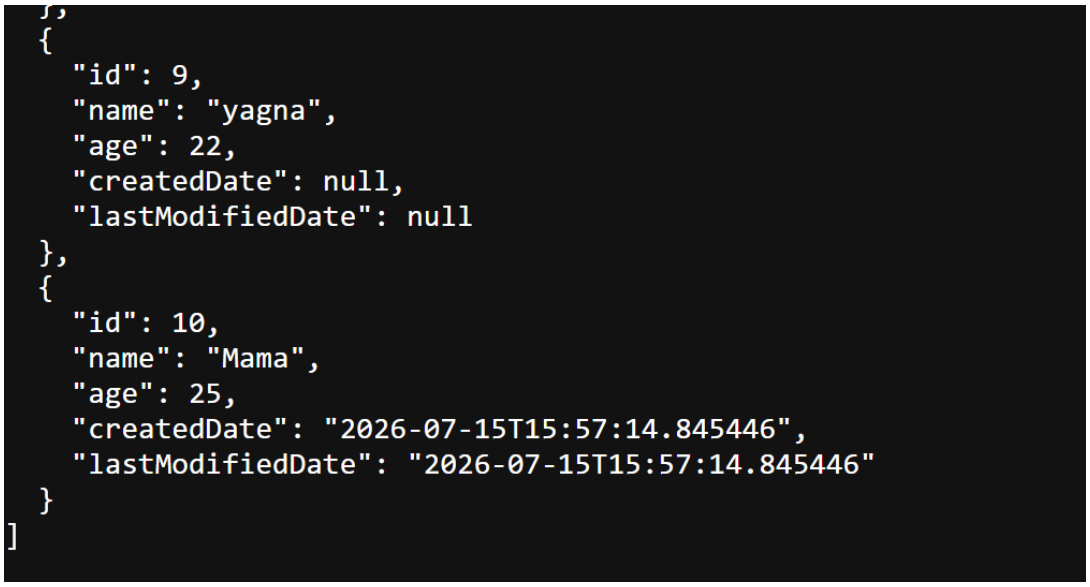
Student.java

```
package Studentmanagement.entity;

import jakarta.persistence.*;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;

@Entity
@Table(name="students")
@EntityListeners(AuditingEntityListener.class)
public class Student {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Integer id;

   private String name;

   private int age;

   @CreatedDate
   @Column(updatable = false)
   private LocalDateTime createdDate;

   @LastModifiedDate
   private LocalDateTime lastModifiedDate;

   public Student() {}

   public Student(String name,int age){
      this.name=name;
      this.age=age;
   }

   public Integer getId() {
      return id;
   }

   public void setId(Integer id) {
      this.id=id;
   }

   public String getName() {
      return name;
```

```
    }

    public void setName(String name) {
        this.name=name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age=age;
    }

    public LocalDateTime getCreatedDate() {
        return createdDate;
    }

    public LocalDateTime getLastModifiedDate() {
        return lastModifiedDate;
    }

}
```

test.http:

```
### Create Student
POST http://localhost:8080/students
Content-Type: application/json

{
  "name": "Mama",
  "age": 25
}s
```

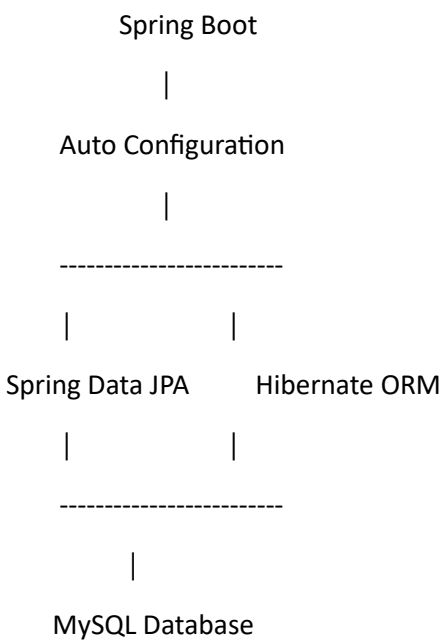


Spring Data JPA and Spring Boot Integration

Spring Boot and Spring Data JPA work together to simplify database-driven application development. Traditionally, developers had to configure database connections, Hibernate, transaction management, and many XML configuration files manually. Spring Boot removes most of this complexity through **auto-configuration**. By adding the required dependencies and configuring a few properties, Spring Boot automatically sets up the DataSource, EntityManager, Transaction Manager, and Hibernate.

Spring Data JPA builds on top of this auto-configuration and provides ready-to-use repository interfaces for performing database operations. As a result, developers can focus on writing business logic instead of spending time on infrastructure configuration. This combination makes Spring Boot one of the most popular frameworks for enterprise Java development.

How Spring Boot Integrates with Spring Data JPA



What is Auto Configuration?

Auto Configuration is a feature of Spring Boot that automatically configures your application based on the dependencies available in the project.

For example, if your project contains:

spring-boot-starter-data-jpa

Spring Boot automatically configures:

- DataSource
- EntityManager
- Hibernate
- Transaction Manager
- Repository Scanning

without requiring any manual configuration.

Before Spring Boot

Developers had to configure everything manually.

Database Connection

↓

Hibernate Configuration

↓

Entity Manager

↓

Transaction Manager

↓

Repositories

This required many XML configuration files.

With Spring Boot

Only two things are needed:

1. Add Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

2. Configure application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/”database name”
spring.datasource.username=”Username”
spring.datasource.password=”Password”
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Spring Boot handles everything else automatically.

Spring Boot Auto Configuration Flow

Application Starts -> Reads pom.xml -> Finds Spring Data JPA -> Creates DataSource -> Creates Hibernate -> SessionFactory -> Creates EntityManager -> Scans @Entity Classes -> Scans JpaRepository Interfaces -> Application Ready

When we start:

```
SpringApplication.run(
    StudentManagementApplication.class,
    args
);
```

Spring Boot automatically:

- Connects to MySQL
- Creates the students table (if it doesn't exist)
- Creates the StudentRepository implementation
- Starts the embedded Tomcat server
- Exposes our REST APIs

Externalizing Configuration

Instead of hardcoding database details in Java files, Spring Boot stores them in:

src/main/resources/application.properties

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=Yagna12
```

This makes it easy to change environments (Development, Testing, Production) without modifying Java code.

Why Use application.properties?

Suppose today your database is:

studentdb

Tomorrow the company changes it to:

companydb

You only change:

```
spring.datasource.url=jdbc:mysql://localhost:3306/companydb
```

No Java code needs to be changed.

Embedded Tomcat

One major advantage of Spring Boot is the embedded web server.

Earlier, developers had to install:

- Apache Tomcat
- Configure WAR files
- Deploy manually

Spring Boot includes Tomcat automatically.

Run:

```
public static void main(String[] args){
   SpringApplication.run(
         StudentManagementApplication.class,
         args
   );
}
```

Console:

Tomcat started on port(s): 8080

Started StudentManagementApplication

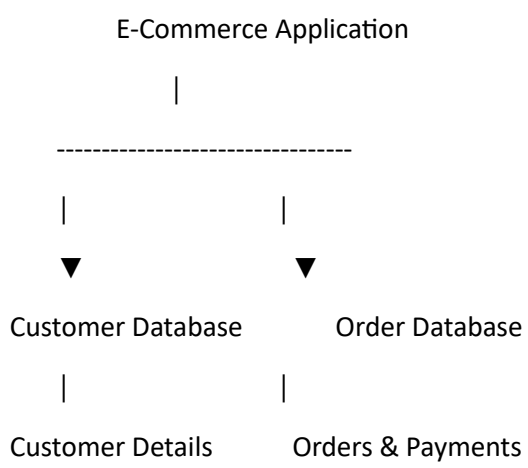
Your application is ready immediately.

Multiple Data Sources in Spring Boot

Most beginner applications use a single database. Spring Boot supports connecting to multiple databases by configuring multiple **DataSources**, **EntityManagers**, and **TransactionManagers**. Each database has its own configuration and repositories. This allows different parts of the application to work with different databases independently.

Why Use Multiple Databases?

Consider an online shopping application.



Keeping data in separate databases improves performance, security, and maintainability.

Single DataSource

Our current project uses only one database.

Spring Boot -> StudentRepository -> studentdb ->

Configuration:

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb

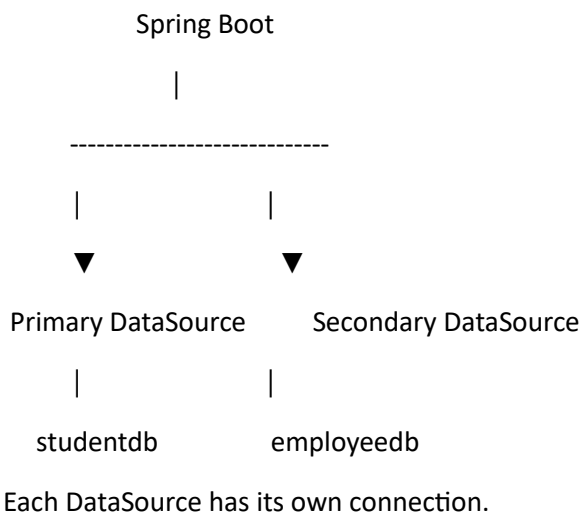
spring.datasource.username=root

spring.datasource.password=Yagna12

This is enough for small and medium-sized applications.

Multiple DataSources

Suppose we need two databases.



Each DataSource has its own connection.

Configuration Example

application.properties

```
# Primary Database
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=Yagna12
# Secondary Database
secondary.datasource.url=jdbc:mysql://localhost:3306/employeedb
secondary.datasource.username=root
secondary.datasource.password=Yagna12
```

Notice that the second database uses a different prefix (secondary.datasource).

Creating Two DataSources

```
@Bean
public DataSource primaryDataSource(){
   return DataSourceBuilder.create().build();
}
@Bean
public DataSource secondaryDataSource(){
   return DataSourceBuilder.create().build();
}
```

Spring Boot now manages two separate database connections.

Spring Data JPA and Hibernate

Hibernate is an **Object Relational Mapping (ORM)** framework used by Spring Data JPA to communicate with relational databases. It converts Java objects into database records and database records back into Java objects. Spring Data JPA provides a simple programming interface, while Hibernate performs the actual database operations behind the scenes.

Hibernate also provides advanced features such as **Lazy Loading**, **Eager Loading**, **Cascade Operations**, **Caching**, and **Batch Processing** to improve application performance and reduce database operations. These features are widely used in enterprise applications.

Hibernate Features

Hibernate provides many features.

- Lazy Loading
- Eager Loading
- Cascade Operations
- Batch Processing
- Caching
- @Transient

1. Lazy Loading

Lazy Loading means ***data is loaded only when it is actually needed***.

Imagine:

Department and Students are the databases we have;

If we fetch a Department, Hibernate loads, Department Only!

Students are **not** loaded immediately.

Only when we call:

department.getStudents();

Hibernate fetches the students.

Example

@OneToMany(fetch = FetchType.LAZY)

private List<Student> students;

2. Eager Loading

Eager Loading means Hibernate ***loads related data immediately.***

Example

@OneToMany(fetch = FetchType.EAGER)

private List<Student> students;

When Department is loaded,

Hibernate automatically loads

- Department
- Students

together.

3. Cascade Operations

Cascade tells Hibernate ***what should happen to related objects.***

Example

Department

↓

Students

If Department is deleted,

Should Students also be deleted?

Cascade decides this.

Example

@OneToMany(

cascade = CascadeType.ALL

)

Now

departmentRepository.delete(id);

also deletes

- Department
- Students

<u>Cascade Types</u>

ALL

CascadeType.ALL

Applies every operation.

PERSIST

CascadeType.PERSIST

Save parent

↓

Save child automatically.

REMOVE

CascadeType.REMOVE

Delete parent

↓

Delete child.

MERGE

CascadeType.MERGE

Update parent

↓

Update child.

REFRESH

Reload data from database.

4. @Transient

Sometimes we don't want a variable stored in the database.

Example

@Transient

private int marks;

Hibernate ignores it.

Example

@Entity

public class Student{

  @Id

  private Integer id;

  private String name;

  @Transient

  private double percentage; // the database won’t have percentage column because we wrote the @transient above

}

Percentage exists only in Java.

5. Batch Processing

Suppose we insert 10,000 students.

Without batching

Insert

Insert

Insert

Insert

10000 Times…

Very slow.

Hibernate Batch Processing

100 Inserts One Batch, 100 Inserts One Batch….

Much faster.

Configuration

spring.jpa.properties.hibernate.jdbc.batch_size=20

Hibernate sends

20 records

at once.

6. Hibernate Cache

Hibernate stores frequently used data in memory.

Instead of

Database

↓

Database

↓

Database

It becomes

Memory Cache

↓

Database (only if needed)

Result

- Faster
- Less database load

JPA Relationships

In real-world applications, database tables are not isolated. They are connected through **relationships**.

For example, one department has many students, one student may enroll in many courses, and one employee has one ID card. Spring Data JPA provides annotations to represent these relationships directly in Java classes, allowing Hibernate to automatically manage the corresponding foreign keys and database tables.

JPA supports four main types of relationships:

- One-to-One (@OneToOne)
- One-to-Many (@OneToMany)
- Many-to-One (@ManyToOne)
- Many-to-Many (@ManyToMany)

These relationships help developers create scalable and maintainable applications without writing complex SQL joins manually.

One-to-One Relationship: One record is associated with exactly **one** record in another table.

Ex: One Student has one ID Card.

StudentManagementApplication.java | StudentController.java | StudentRepository.java

```java
package Studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@SpringBootApplication
@EnableJpaAuditing
public class StudentManagementApplication {

    public static void main(String[] args) {

        SpringApplication.run(StudentManagementApplication.class,args);

    }

}
```

StudentManagementApplication.java | StudentController.java | StudentRepositor

```java
package Studentmanagement.controller;

import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/people")
public class StudentController {

    private final StudentRepository repository;  3 usages

    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Student save(@RequestBody Student student){
        return repository.save(student);
    }

    @GetMapping
    public List<Student> getAll(){
        return repository.findAll();
    }

}
```

StudentManagementApplication.java | StudentController.java | StudentRepository.java

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student,Integer> {
}
```

StudentManagementApplication.java | StudentController.java | StudentRepository.java | Student.java

```java
package Studentmanagement.entity;
import jakarta.persistence.*;
@Entity  6 usages
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;  3 usages
    @OneToOne(cascade = CascadeType.ALL)  3 usages
    @JoinColumn(name = "card_id")
    private IDCard card;
    public Student() {}

    public Student(String name, IDCard card) {  no usages
        this.name = name;
        this.card = card;
    }

    public Integer getId() {  no usages
        return id;
    }

    public void setId(Integer id) {  no usages
        this.id = id;
    }

    public String getName() {  no usages
        return name;
    }

    public void setName(String name) {  no usages
        this.name = name;
    }

    public IDCard getCard() {  no usages
        return card;
    }

    public void setCard(IDCard card) {  no usages
        this.card = card;
    }
}
```

StudentManagementApplication.java | StudentController.java | StudentRepository.java | Student.java | IDCard.java

```java
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity  4 usages
public class IDCard {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String cardNumber;  3 usages

    public IDCard() {}

    public IDCard(String cardNumber) {  no usages
        this.cardNumber = cardNumber;
    }

    public Integer getId() {  no usages
        return id;
    }

    public String getCardNumber() {  no usages
        return cardNumber;
    }

    public void setCardNumber(String cardNumber) {  no usages
        this.cardNumber = cardNumber;
    }
}
```

Run with: No Environment

```
POST http://localhost:8080/people
Content-Type: application/json

{
  "name": "Yagna",
  "card": {
    "cardNumber": "AP2026001"
  }
}
```

localhost:8080/people

Pretty-print

```
[
  {
    "id": 1,
    "name": "Yagna",
    "card": {
      "id": 1,
      "cardNumber": "AP2026001"
    }
  }
]
```

One-to-Many Relationship: One parent record is related to many child records.

Ex: One Department has many Students.

Student.java × DepartmentRepository.java DepartmentController.java

```java
package Studentmanagement.entity;
import jakarta.persistence.*;

@Entity  9 usages
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;  2 usages

    public Student() {
    }

    public Integer getId() {  no usages
        return id;
    }

    public void setId(Integer id) {  no usages
        this.id = id;
    }

    public String getName() {  no usages
        return name;
    }

    public void setName(String name) {  no usages
        this.name = name;
    }
}
```

Student.java DepartmentRepository.java × DepartmentController.java

```java
package Studentmanagement.repository;
import Studentmanagement.entity.Department;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository  no usages
        extends JpaRepository<Department, Integer> {
}
```

Student.java | DepartmentRepository.java | DepartmentController.java

```java
package Studentmanagement.controller;
import Studentmanagement.entity.Department;
import Studentmanagement.repository.DepartmentRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/department")
public class DepartmentController {

    private final DepartmentRepository repository;  3 usages

    public DepartmentController(DepartmentRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Department save(@RequestBody Department department) {
        return repository.save(department);
    }

    @GetMapping
    public List<Department> getAll() {
        return repository.findAll();
    }
}
```

Student.java | DepartmentRepository.java | DepartmentController.java | StudentManagementApplication.java

```java
package Studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentManagementApplication.class, args);
    }
}
```

Student.java | DepartmentRepository.java | DepartmentController.java | StudentManagementApplication.java | Department.java

```java
package Studentmanagement.entity;
import jakarta.persistence.*;
import java.util.List;

@Entity 6 usages
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String departmentName; 2 usages

    @OneToMany(cascade = CascadeType.ALL) 2 usages
    @JoinColumn(name = "department_id")
    private List<Student> students;

    public Department() {
    }

    public Integer getId() { no usages
        return id;
    }

    public void setId(Integer id) { no usages
        this.id = id;
    }

    public String getDepartmentName() { no usages
        return departmentName;
    }

    public void setDepartmentName(String departmentName) { no usages
        this.departmentName = departmentName;
    }

    public List<Student> getStudents() { no usages
        return students;
    }

    public void setStudents(List<Student> students) { no usages
        this.students = students;
    }
}
```

Run with: No Environment

```
POST http://localhost:8080/department
Content-Type: application/json

{
  "departmentName": "CSE",
  "students": [
    {
      "name": "Yagna"
    },
    {
      "name": "Rahul"
    },
    {
      "name": "Priya"
    }
  ]
}
```

localhost:8080/department

Pretty-print


```
[
  {
    "id": 1,
    "departmentName": "CSE",
    "students": [
      {
        "id": 2,
        "name": "Yagna"
      },
      {
        "id": 3,
        "name": "Rahul"
      },
      {
        "id": 4,
        "name": "Priya"
      }
    ]
  }
]
```

Many-to-One Relationship: This is the opposite side of the previous relationship.

Ex: Many Students belong to one Department.

Student.java | DepartmentRepository.java | StudentRepository.java

```java
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    public Student() {
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Department getDepartment() {
        return department;
    }

    public void setDepartment(Department department) {
        this.department = department;
    }
}
```

Student.java | DepartmentRepository.java | StudentRepository.java

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Department;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepartmentRepository  3 usages
        extends JpaRepository<Department,Integer> {
}
```

ntRepository.java × StudentRepository.java × DepartmentController.

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository  3 usages
        extends JpaRepository<Student,Integer> {
}
```

StudentRepository.java × DepartmentController.java × StudentController.java

```java
package Studentmanagement.controller;

import Studentmanagement.entity.Department;
import Studentmanagement.repository.DepartmentRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/departments")
public class DepartmentController {

    private final DepartmentRepository repository;  3 usages

    public DepartmentController(DepartmentRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Department save(@RequestBody Department department){
        return repository.save(department);
    }

    @GetMapping
    public List<Department> getAll(){
        return repository.findAll();
    }
}
```

DepartmentController.java | StudentController.java | StudentManagem

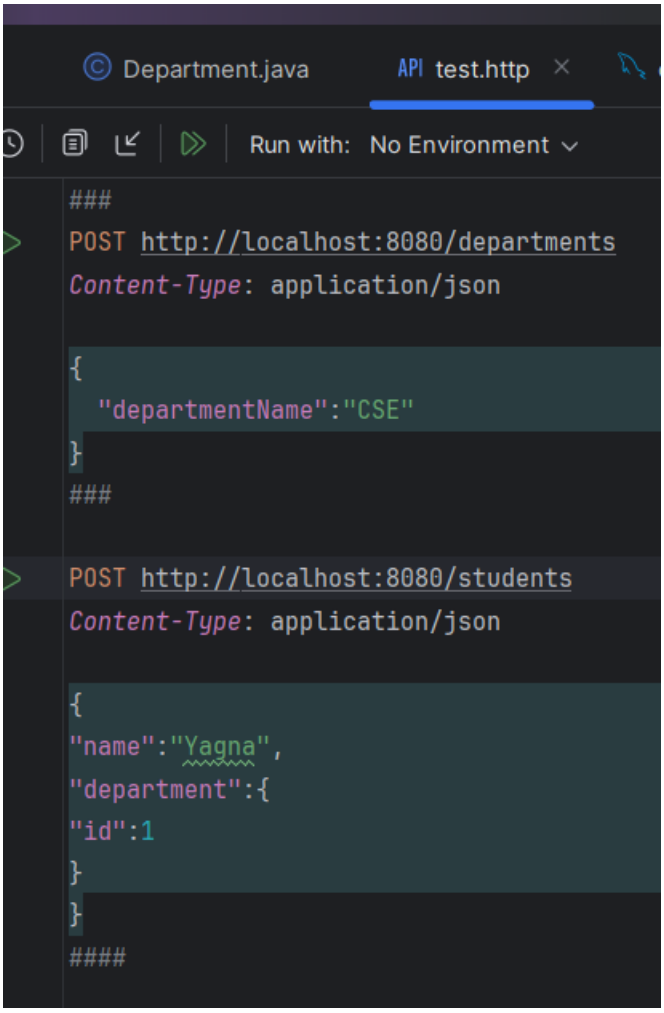
```java
package Studentmanagement.controller;

import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentRepository repository;  3 usages

    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Student save(@RequestBody Student student){
        return repository.save(student);
    }

    @GetMapping
    public List<Student> getAll(){
        return repository.findAll();
    }
}
```

dentController.java | StudentManagementApplication.java | Department.java

```java
package Studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentManagementApplication.class, args);
    }
}
```

StudentManagementApplication.java | Department.java | test.http

```java
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity  6 usages
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String departmentName;  2 usages

    public Department() {
    }

    public Integer getId() {  no usages
        return id;
    }

    public void setId(Integer id) {  no usages
        this.id = id;
    }

    public String getDepartmentName() {  no usages
        return departmentName;
    }

    public void setDepartmentName(String departmentName) {  no usages
        this.departmentName = departmentName;
    }
}
```


Department.java
API test.http
Run with: No Environment
###
POST http://localhost:8080/departments
Content-Type: application/json
{
"departmentName":"CSE"
}
###
POST http://localhost:8080/students
Content-Type: application/json
{
"name":"Yagna",
"department":{
"id":1
}
}
####


localhost:8080/departments
Pretty-print
[
{
"id": 1,
"departmentName": "CSE"
},
{
"id": 2,
"departmentName": "CSE"
}
]


localhost:8080/students
Pretty-print
[
{
"id": 1,
"name": "Yagna",
"department": null
},
{
"id": 2,
"name": "Yagna",
"department": {
"id": 1,
"departmentName": "CSE"
}
},
{
"id": 3,
"name": "Rahul",
"department": {
"id": 1,
"departmentName": "CSE"
}
},
{
"id": 4,
"name": "Priya",
"department": {
"id": 1,
"departmentName": "CSE"
}
},
{
"id": 5,
"name": "Yagna",
"department": {
"id": 1,
"departmentName": "CSE"
}
}
]

Many-to-Many Relationship: Many records in one table are related to many records in another table.

Ex: Students and Courses.

A student can study multiple courses.

A course can have many students.

StudentRepository.java

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository  3 usages
        extends JpaRepository<Student,Integer> {
}
```

Student.java

```java
package Studentmanagement.entity;
import jakarta.persistence.*;
import java.util.List;
@Entity  6 usages
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;  2 usages
    @ManyToMany(cascade = CascadeType.ALL)  2 usages

    @JoinTable(
            name = "student_course",

            joinColumns = @JoinColumn(name="student_id"),

            inverseJoinColumns = @JoinColumn(name="course_id")
    )
    private List<Course> courses;
    public Student() {
    }

    public Integer getId() {  no usages
        return id;
    }

    public void setId(Integer id) {  no usages
        this.id=id;
    }

    public String getName() {  no usages
        return name;
    }

    public void setName(String name) {  no usages
        this.name=name;
    }

    public List<Course> getCourses() {  no usages
        return courses;
    }

    public void setCourses(List<Course> courses) {  no usages
        this.courses=courses;
    }
}
```

StudentRepository.java | Student.java | StudentController.java

```java
package Studentmanagement.controller;

import Studentmanagement.entity.Student;
import Studentmanagement.repository.StudentRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentRepository repository;  3 usages

    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Student save(@RequestBody Student student){

        return repository.save(student);

    }

    @GetMapping
    public List<Student> getAll(){

        return repository.findAll();

    }

}
```

StudentController.java | StudentManagementApplication.java | API tes

```java
package Studentmanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentManagementApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentManagementApplication.class, args);
    }
}
```

agementApplication.java API test.http console_1 [M

Run with: No Environment

```
POST http://localhost:8080/students
Content-Type: application/json

{
  "name":"Yagna",

  "courses":[

    {
      "courseName":"Java"
    },

    {
      "courseName":"Spring Boot"
    },

    {
      "courseName":"DBMS"
    }

  ]

}
####
```

[MySQL_root_Yagna12] Course.java CourseRepository.java

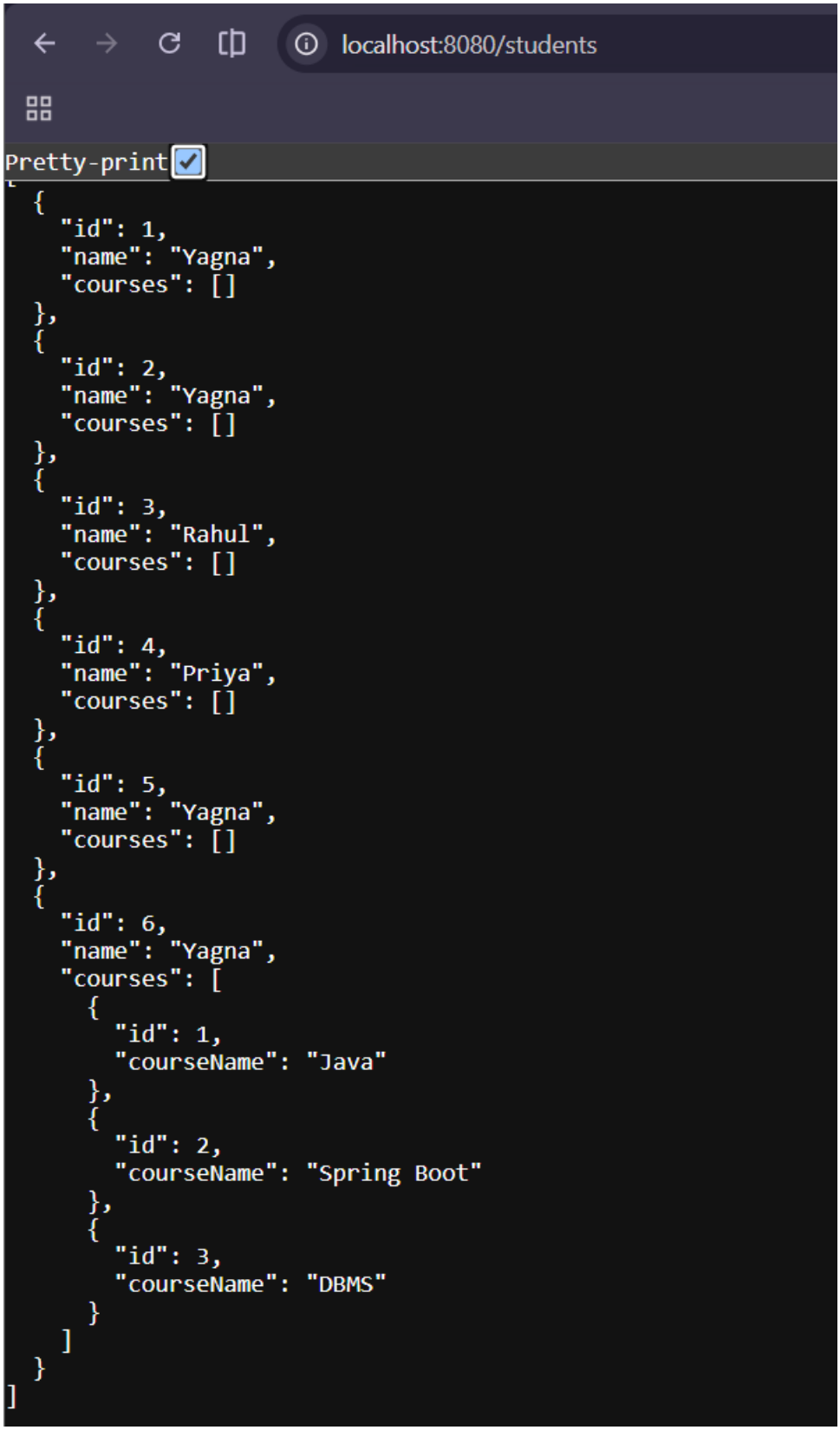
```
package Studentmanagement.entity;

import jakarta.persistence.*;

@Entity  3 usages
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String courseName;  2 usages

    public Course() {
    }

    public Integer getId() {  no usages
        return id;
    }

    public void setId(Integer id) {  no usages
        this.id=id;
    }

    public String getCourseName() {  no usages
        return courseName;
    }

    public void setCourseName(String courseName) {  no usage
        this.courseName=courseName;
    }

}
```

na12] Course.java CourseRepository.java × CourseController.

```java
package Studentmanagement.repository;

import Studentmanagement.entity.Course;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CourseRepository  no usages
        extends JpaRepository<Course,Integer> {
}
```

agna12] Course.java CourseRepository.java CourseController.java ×

```java
package Studentmanagement.controller;

import Studentmanagement.entity.Course;
import Studentmanagement.repository.CourseRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/courses")
public class CourseController {

    private final CourseRepository repository;  3 usages

    public CourseController(CourseRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public Course save(@RequestBody Course course){

        return repository.save(course);

    }

    @GetMapping
    public List<Course> getAll(){

        return repository.findAll();

    }

}
```




```
[
  {
    "id": 1,
    "name": "Yagna",
    "courses": []
  },
  {
    "id": 2,
    "name": "Yagna",
    "courses": []
  },
  {
    "id": 3,
    "name": "Rahul",
    "courses": []
  },
  {
    "id": 4,
    "name": "Priya",
    "courses": []
  },
  {
    "id": 5,
    "name": "Yagna",
    "courses": []
  },
  {
    "id": 6,
    "name": "Yagna",
    "courses": [
      {
        "id": 1,
        "courseName": "Java"
      },
      {
        "id": 2,
        "courseName": "Spring Boot"
      },
      {
        "id": 3,
        "courseName": "DBMS"
      }
    ]
  }
]
```